

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ТЕОРИЯ

ЧТО ТАКОЕ ФОНОВЫЙ ЦИКЛ?

В любой игре что-то происходит постоянно, даже когда игрок ничего не нажимает. Персонажи двигаются, монстры атакуют, здоровье уменьшается - всё это работает в фоне.

Фоновый цикл (game loop) - это бесконечный цикл, который выполняется много раз в секунду. Каждый 'тик' цикла игра обновляет состояние: перемещает объекты, проверяет столкновения, начисляет очки.

В наших предыдущих уроках мы использовали `LaunchedEffect` с `while(true)` для создания фоновых процессов. Это простой и эффективный способ!

УПРАВЛЕНИЕ РИСКАМИ В ИГРАХ

Хорошая игра должна быть не только веселой, но и требовать стратегии. Механика риска - это когда игрок должен выбирать между выгодой и опасностью.

В нашем реакторе игрок получает энергию, но при этом растет температура. Если перегреть реактор - теряется половина энергии! Игрок должен вовремя охлаждать систему или остановить сборку.

Такие механики заставляют игрока думать и планировать, а не просто нажимать кнопки.

ШАБЛОНЫ КОДА: DATA CLASS

`data class` - это специальный класс в Kotlin для хранения данных. Он автоматически создает полезные функции: `equals()`, `hashCode()`, `toString()` и `copy()`.

Мы используем `data class` для описания улучшений в магазине. Каждое улучшение имеет имя, цену, уровень и иконку.

Свойство с `get()` - это вычисляемое свойство. Оно не хранится напрямую, а считается по формуле каждый раз, когда мы к нему обращаемся.

МОДУЛЬНЫЙ UI: КОМПОНЕНТЫ

Когда экран становится сложным, код может превратиться в кашу из скобок. Чтобы этого избежать, мы делим интерфейс на маленькие независимые части - компоненты.

Каждый компонент - это отдельная `@Composable` функция. Она принимает данные как параметры и рисует свою часть экрана.

В этом уроке мы создадим `UpgradeRow` - отдельный компонент для одной строки в магазине. Тогда код главного экрана станет чище.

КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 1: 1. Модель данных для магазина (СНАРУЖИ экрана)

```
// Чертеж карточки магазина. Пишем в самом верху файла.  
data class UpgradeItem(  
    val id: String,  
    val name: String,  
    val description: String,  
    val baseCost: Long,  
    val costMultiplier: Double,  
    val level: Int,  
    val icon: String  
) {  
    val currentCost: Long  
    get() = (baseCost * kotlin.math.pow(costMultiplier, level.toDouble())).toLong()  
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 2: 2. Компонент UI: Строка Улучшения (UpgradeRow)

```
// Пишем СНАРУЖИ главного экрана.
@Composable
fun UpgradeRow(upgrade: UpgradeItem, canAfford: Boolean, onBuyClick: () -> Unit)
{
    Card(
        modifier = Modifier.fillMaxWidth(),
        colors = CardDefaults.cardColors(containerColor = Color(0xFF131A2B))
    ) {
        Row(
            modifier = Modifier.fillMaxWidth().padding(12.dp),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column {
                Text(upgrade.icon + " " + upgrade.name, color = Color.White, fontWeight =
                FontWeight.Bold)
                Text(upgrade.description, color = Color.Gray, fontSize = 11.sp)
                Text("Ур. " + upgrade.level, color = Color(0xFF00FFCC), fontSize = 11.sp)
            }
            Button(onClick = onBuyClick, enabled = canAfford) {
                Text(upgrade.currentCost.toString() + " E")
            }
        }
    }
}
```

ЭТАП 3: 3. Объявляем главный экран (Голова)

```
@Composable
fun SolarHarvesterScreen() {
    // ВСЬ КОД ИЗ СЛЕДУЮЩИХ ШАГОВ ПИШЕМ ТУТ, ВНУТРИ ЭТОЙ ФУНКЦИИ
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 4: 4. Переменные состояния (Память реактора)

```
var energy by remember { mutableStateOf(0L) }
var isRunning by remember { mutableStateOf(false) }
var heatLevel by remember { mutableStateOf(0f) }

var solarLevel by remember { mutableStateOf(1) }
var coolingLevel by remember { mutableStateOf(1) }
var batteryLevel by remember { mutableStateOf(1) }
```

ЭТАП 5: 5. Динамическая математика (Характеристики)

```
val energyPerTick = solarLevel * 8L
val heatPerTick = 0.04f / (1f + (coolingLevel * 0.15f))
val maxCapacity = batteryLevel * 400L
```

ЭТАП 6: 6. Автоматический реактор (LaunchedEffect)

```
LaunchedEffect(isRunning) {
    if (isRunning) {
        while (isRunning) {
            delay(300)
            heatLevel += heatPerTick

            if (heatLevel >= 1.0f) {
                isRunning = false
                energy /= 2
                heatLevel = 0f
            }

            if (energy < maxCapacity) {
                energy = (energy + energyPerTick).coerceAtMost(maxCapacity)
            }
        }
    }
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 7: 7. Основной контейнер экрана (Box)

```
Box(  
    modifier = Modifier.fillMaxSize().background(Color(0xFF070B12)),  
    contentAlignment = Alignment.Center  
) {  
    Column(  
        modifier = Modifier.fillMaxSize().padding(16.dp),  
        horizontalAlignment = Alignment.CenterHorizontally,  
        verticalArrangement = Arrangement.SpaceBetween  
    ) {  
        /* СЮДА БУДЕМ ВКЛАДЫВАТЬ ТЕКСТЫ И КНОПКИ */  
    }  
}
```

ЭТАП 8: 8. Панель мониторов (Dashboard)

```
Column(horizontalAlignment = Alignment.CenterHorizontally) {  
    Text("КВАНТОВЫЙ РЕАКТОР", color = Color.Gray, fontSize = 12.sp)  
    Spacer(Modifier.height(8.dp))  
    Text("$energy / $maxCapacity E", color = Color(0xFF00FFCC), fontSize = 32.sp,  
        fontWeight = FontWeight.Bold)  
    Spacer(Modifier.height(16.dp))  
    Text("Температура ядра:", color = Color.White, fontSize = 12.sp)  
    LinearProgressIndicator(  
        progress = { heatLevel },  
        modifier = Modifier.fillMaxWidth().height(12.dp).clip(RoundedCornerShape(6.dp)),  
        color = if (heatLevel > 0.75f) Color.Red else Color(0xFF00FFCC)  
    )  
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 9: 9. Панель ручного контроля

```
Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement =
Arrangement.spacedBy(8.dp)) {
    Button(
        onClick = { isRunning = !isRunning },
        modifier = Modifier.weight(1f).height(50.dp),
        colors = ButtonDefaults.buttonColors(
containerColor = if (isRunning) Color(0xFFD32F2F) else Color(0xFF388E3C)
        )
    ) { Text(if (isRunning) "СТОП" else "СТАРТ", fontWeight = FontWeight.Bold) }
    Button(
        onClick = { heatLevel = (heatLevel - 0.15f).coerceAtLeast(0f) },
        modifier = Modifier.weight(1f).height(50.dp),
colors = ButtonDefaults.buttonColors(containerColor = Color(0xFF1976D2)),
        enabled = heatLevel > 0f
    ) { Text("ОХЛАДИТЬ", fontWeight = FontWeight.Bold) }
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ЭТАП 10: 10. Магазин Улучшений через LazyColumn

```
        val upgrades = listOf(
            UpgradeItem("solar", "Солнечные Панели", "+$energyPerTick Е/тик", 100L, 1.4,
                solarLevel, "☀ "),
            UpgradeItem("heatsink", "Охлаждение радиатора", "-15% нагрева", 150L, 1.3,
                coolingLevel, "❄ "),
            UpgradeItem("battery", "Емкость Батареи", "+400 макс. энергии", 200L, 1.5,
                batteryLevel, " ")
        )

        LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp), modifier =
            Modifier.fillMaxWidth().height(260.dp)) {
            items(upgrades) { item ->
                UpgradeRow(
                    upgrade = item,
                    canAfford = energy >= item.currentCost,
                    onBuyClick = {
                        energy -= item.currentCost
                        when (item.id) {
                            "solar" -> solarLevel += 1
                            "heatsink" -> coolingLevel += 1
                            "battery" -> batteryLevel += 1
                        }
                    }
                )
            }
        }
    }
```

ЭТАП 11: 11. Предпросмотр Экрана (Preview)

```
// Пишется СНАРУЖИ основной функции SolarHarvesterScreen!
@Preview
@Composable
fun SolarHarvesterPreview() {
    MaterialTheme {
        SolarHarvesterScreen()
    }
}
```


КОСМИЧЕСКАЯ МИССИЯ: КВАНТОВЫЙ РЕАКТОР

МОДУЛЬ: ФОНОВЫЕ ЦИКЛЫ, ШАБЛОНЫ И УПРАВЛЕНИЕ РИСКАМИ | ID: 5

ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ:

Добавь Систему Сверхзаряда: создай переменную superchargeChance. Сделай так, чтобы с шансом 15% (используй Random) ручное охлаждение давало мгновенный бонус +50 энергии вместо обычного сброса тепла.